

Assignment 3.2: Real-World Optimization with PuLP

Name Gunasekaran pasupathy **Date** 7/12/2026 - 7/14/2026 **Course:** AAI 501 — Module 3

The problem in my own words: I run a factory making two products. A makes 40/unit, B makes 60/unit. Both need time on two machines, and machine hours are limited. I can optionally pay \$500 to run Machine 2 on Saturday for 20 extra hours, and I have to make at least 10 units of B per week. Goal: pick production amounts (and the overtime decision) to maximize weekly profit.

Given info:

	Machine 1 (hrs/unit)	Machine 2 (hrs/unit)	Profit/unit
Product A	2	3	\$40
Product B	4	2	\$60

- Machine 1: 100 hours/week available
- Machine 2: 80 hours/week, **plus 20 more if I pay \$500 for Saturday overtime**
- Must produce at least 10 units of B

Decision variables:

- x = units of Product A (continuous, ≥ 0)
- y = units of Product B (continuous)
- z = 1 if we run Machine 2 overtime on Saturday, 0 if not (binary)

Part 1: Objective function

Profit is 40 per unit of A plus 60 per unit of B, minus the 500 overtime fee **only if** we actually use overtime (that's what multiplying by the binary z does — if $z = 0$ the fee disappears):

$$\text{Maximize } Z = 40x + 60y - 500z$$

Part 2: Constraints

Machine 1 hours — each A takes 2 hrs, each B takes 4 hrs, 100 hrs available:

$$2x + 4y \leq 100$$

Machine 2 hours — each A takes 3 hrs, each B takes 2 hrs. Base is 80 hrs, and overtime adds 20 more but only when $z = 1$. So the right side depends on z (this is the same on/off trick I practiced in the lab):

$$3x + 2y \leq 80 + 20z$$

Minimum quota for B:

$$y \geq 10$$

Variable domains:

$$x \geq 0, \quad y \geq 0, \quad z \in \{0, 1\}$$

(The $y \geq 10$ quota already forces $y \geq 0$, but I'm listing the domains for completeness.)

Part 3: Solving with PuLP

```
In [1]: %pip install pulp
```

Defaulting to user installation because normal site-packages is not writeable

Requirement already satisfied: pulp in /sessions/kind-awesome-carson/.local/lib/python3.10/site-packages (3.3.2)

```
In [2]: import pulp
```

```
# maximization problem
factory = pulp.LpProblem("Factory_Weekly_Profit", pulp.LpMaximize)
```

```
In [3]: # decision variables
x = pulp.LpVariable('x', lowBound=0, cat='Continuous') # units of A
y = pulp.LpVariable('y', lowBound=0, cat='Continuous') # units of B
z = pulp.LpVariable('z', cat='Binary')                # 1 = run Saturday
```

```
In [4]: # objective: profit from A and B, minus $500 if we use overtime
factory += 40*x + 60*y - 500*z, "Total_Profit"

# Machine 1: 2 hrs per A + 4 hrs per B, 100 hrs available
factory += 2*x + 4*y <= 100, "Machine_1_hours"

# Machine 2: 3 hrs per A + 2 hrs per B, 80 hrs (+20 more only if z = 1)
factory += 3*x + 2*y <= 80 + 20*z, "Machine_2_hours"

# must make at least 10 units of B
factory += y >= 10, "Min_B_quota"

factory # printing the model to double-check it matches my math above
```

```
Out[4]: Factory_Weekly_Profit:
MAXIMIZE
40*x + 60*y + -500*z + 0
SUBJECT TO
Machine_1_hours: 2 x + 4 y <= 100

Machine_2_hours: 3 x + 2 y - 20 z <= 80

Min_B_quota: y >= 10

VARIABLES
x Continuous
y Continuous
0 <= z <= 1 Integer
```

```
In [5]: factory.solve()
print("Status:", pulp.LpStatus[factory.status]) # checking status BEFORE tr
```

Welcome to the CBC MILP Solver

Version: 2.10.10

Build Date: Sep 26 2023

command line - /sessions/kind-awesome-carson/.local/lib/python3.10/site-packages/pulp/apis/./solverdir/cbc/linux/arm64/cbc /sessions/kind-awesome-carson/tmp/53c774ad24c2401883f2e9be24bf0507-pulp.mps -max -timeMode elapsed -solve -printingOptions all -solution /sessions/kind-awesome-carson/tmp/53c774ad24c2401883f2e9be24bf0507-pulp.sol (default strategy 1)

At line 2 NAME MODEL

At line 3 ROWS

At line 8 COLUMNS

At line 20 RHS

At line 24 BOUNDS

At line 26 ENDATA

Problem MODEL has 3 rows, 3 columns and 6 elements

Coin0008I MODEL read with 0 errors

Option for timeMode changed from cpu to elapsed

Continuous objective value is 1650 - 0.00 seconds

Cgl0004I processed model has 2 rows, 3 columns (1 integer (1 of which binary)) and 5 elements

Cbc0038I Initial state - 0 integers unsatisfied sum - 0

Cbc0038I Solution found of 1650

Cbc0038I Relaxing continuous gives 1650

Cbc0038I Before mini branch and bound, 1 integers at bound fixed and 0 continuous

Cbc0038I Mini branch and bound did not improve solution (0.00 seconds)

Cbc0038I After 0.00 seconds - Feasibility pump exiting with objective of 1650 - took 0.00 seconds

Cbc0012I Integer solution of 1650 found by feasibility pump after 0 iterations and 0 nodes (0.00 seconds)

Cbc0001I Search completed - best objective 1650, took 0 iterations and 0 nodes (0.00 seconds)

Cbc0035I Maximum depth 0, 0 variables fixed on reduced cost

Cuts at root node changed objective from 1650 to 1650

Probing was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

Gomory was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

Knapsack was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

Clique was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

MixedIntegerRounding2 was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

FlowCover was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

TwoMirCuts was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

ZeroHalf was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

Result - Optimal solution found

Objective value: 1650.00000000

Enumerated nodes: 0

```
Total iterations:          0
Time (CPU seconds):       0.00
Time (Wallclock seconds): 0.00

Option for printingOptions changed from normal to all
Total time (CPU seconds):  0.00   (Wallclock seconds):    0.00

Status: Optimal
```

```
In [6]: print("Product A (x):", x.varValue, "units")
        print("Product B (y):", y.varValue, "units")
        print("Overtime (z): ", z.varValue, "-> run Saturday overtime" if z.varValue
        print("Max weekly profit: $", pulp.value(factory.objective))
```

```
Product A (x): 15.0 units
Product B (y): 17.5 units
Overtime (z):  0.0 -> NO overtime
Max weekly profit: $ 1650.0
```

Checking the answer by hand

The solver says no overtime ($z = 0$), make 15 units of A and 17.5 units of B. Let me verify the constraints are satisfied and see which ones are binding:

```
In [7]: m1_used = 2*x.varValue + 4*y.varValue
        m2_used = 3*x.varValue + 2*y.varValue
        print(f"Machine 1: {m1_used} of 100 hrs used")
        print(f"Machine 2: {m2_used} of {80 + 20*z.varValue:.0f} hrs used")
        print(f"B quota:   {y.varValue} >= 10 ok")
        print(f"Profit:    40*{x.varValue} + 60*{y.varValue} - 500*{z.varValue:.0f}")
```

```
Machine 1: 100.0 of 100 hrs used
Machine 2: 80.0 of 80 hrs used
B quota:   17.5 >= 10 ok
Profit:    40*15.0 + 60*17.5 - 500*0 = 1650.0
```

Both machines are used up completely (100/100 and 80/80), so the optimum sits exactly where the two machine constraints intersect — same "corner point" idea from the lab.

But why isn't overtime worth it? \$500 for 20 extra hours sounded plausible. To convince myself, I'll force $z = 1$ and re-solve to see what the extra hours would actually earn:

```
In [8]: # force overtime ON and see the best possible profit
        test = pulp.LpProblem("Force_Overtime", pulp.LpMaximize)
        xt = pulp.LpVariable('xt', lowBound=0)
        yt = pulp.LpVariable('yt', lowBound=0)

        test += 40*xt + 60*yt - 500           # z fixed at 1, so fee is always paid
        test += 2*xt + 4*yt <= 100
        test += 3*xt + 2*yt <= 100           # 80 + 20 overtime hours
        test += yt >= 10
```

```
test.solve(pulp.PULP_CBC_CMD(msg=0))
print("With overtime forced ON:")
print("  A =", xt.varValue, ", B =", yt.varValue)
print("  Profit = $", pulp.value(test.objective))
print()
print("Extra revenue from the 20 hrs:", 40*xt.varValue + 60*yt.varValue - 16
```

With overtime forced ON:

A = 25.0 , B = 12.5

Profit = \$ 1250.0

Extra revenue from the 20 hrs: 100.0 → way less than the \$500 fee

So the 20 extra Machine 2 hours would only generate 100 of additional revenue (profit goes from 1650 to 1750 *before* the fee), because Machine 1 becomes the bottleneck — I run out of Machine 1 hours before I can use all the new Machine 2 time. Paying 500 to gain 100 is a bad trade, so the solver correctly says $z = 0$. That works out to 5 of value per extra Machine 2 hour, so overtime would only make sense if it cost less than about 100.

Final Answers

1) Objective function: Maximize $Z = 40x + 60y - 500z$

2) Constraints:

- Machine 1: $2x + 4y \leq 100$
- Machine 2: $3x + 2y \leq 80 + 20z$
- Quota: $y \geq 10$
- $x, y \geq 0$, z binary

3) Optimal solution:

- Produce **15 units of Product A** and **17.5 units of Product B** per week
- **Do NOT run Machine 2 overtime** ($z = 0$) — the extra hours would only add ~ 100 of profit, far less than the 500 cost
- **Maximum weekly profit: \$1,650**

(Since the problem defines x and y as continuous, the 17.5 units of B is fine — in a real factory that could just mean a half-finished unit carrying into next week.)

*AI disclosure: I used (ChatGpt 5.6)) as an assistant to format markdown syntax in some places to display text better. The model setup, the math, and the interpretation of results reflect my own understanding from the Module 3 lab. *